

Introduction à la conteneurisation & construire un conteneur manuellement

Chapitre 01 — Fondations Linux : kernel, processus, filesystem, namespaces, cgroups

Version de révision : schémas explicatifs · termes expliqués · points clés · corrigé des questions

Objectif

Comprendre les mécanismes fondamentaux de Linux qui permettent aux conteneurs de fonctionner, **avant** d'étudier leur construction.

Idée clé

Un conteneur **n'est pas une machine virtuelle**. C'est essentiellement un **processus Linux ordinaire** (ou un groupe de processus) **isolé** grâce aux **namespaces** et **limité** grâce aux **cgroups**, le tout géré par le **noyau Linux**.



La recette complète d'un conteneur (détaillée dans ce chapitre).

Comment lire ce document

Bloc	Signification
En clair	Explication simple d'un terme important, avec une analogie.
À retenir	Les points clés de la section, à connaître par cœur.
Attention	Pièges, confusions fréquentes et nuances importantes.
Blocs sombres	Commandes à tester dans un terminal Linux avec Docker installé.

Sommaire

0. Pourquoi commencer par Linux ?	3
1. Processus	3
2. Le Kernel Linux	4
3. Filesystem	6
4. Root Filesystem : rootfs	6
5. Mount	7
6. chroot	8
7. pivot_root	9
8. Les Namespaces Linux	9
9. Cgroups	13
10. Exemple complet : lancement d'un conteneur	15
Ce qu'il faut absolument retenir	16
Glossaire des termes importants	17
Questions de compréhension — avec corrigés	18
Préparation pour la suite	19

0. Pourquoi commencer par Linux ?

Un conteneur n'est pas une machine virtuelle complète. Il s'appuie sur les mécanismes fournis par le **kernel Linux** pour :

- créer et gérer des **processus** ;
- isoler les processus ;
- contrôler les **ressources** ;
- gérer les **systèmes de fichiers** ;
- gérer le **réseau** ;
- contrôler les **permissions** et les **capacités**.

L'idée fondamentale est donc la chaîne suivante : une application ne parle jamais directement au matériel, elle passe toujours par le kernel.

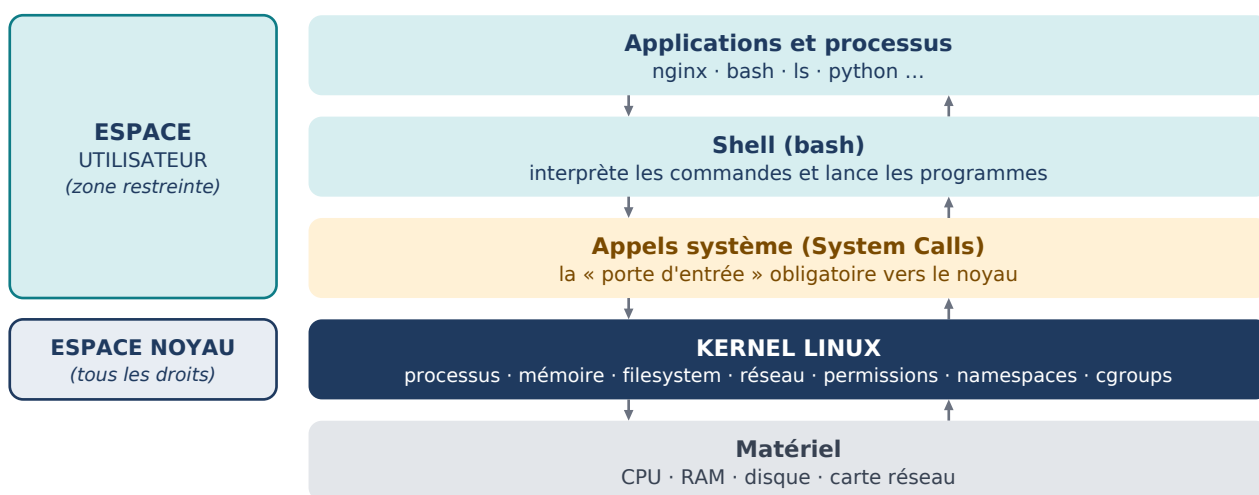


Figure 1 — Du programme au matériel : tout passe par les appels système et le kernel.

Comprendre cette chaîne permet de comprendre ensuite ce que font réellement les runtimes de conteneurs.

À retenir

- ✓ Les conteneurs ne « contiennent » pas un OS : ils utilisent les **fonctions du kernel Linux** de l'hôte.
- ✓ Toute opération (fichier, réseau, processus, mémoire) passe par le kernel.

1. Processus

Un **processus** est simplement un **programme en cours d'exécution**. Par exemple :

```
sleep 100 &
```

Lorsque cette commande s'exécute, Linux crée un processus correspondant au programme `sleep`. On peut voir les processus avec :

```
ps
ps aux
top
```

Chaque processus possède un identifiant appelé **PID** (*Process ID*). Le PID permet au kernel et aux outils Linux d'identifier les processus :

PID	COMMAND
1	systemd
1200	sshd
2450	bash
3100	nginx

```
top -p PID
```

En clair — Programme vs processus

Un **programme** est une recette de cuisine écrite dans un livre (un fichier sur le disque). Un **processus** est le cuisinier qui suit la recette en ce moment (le programme chargé en mémoire et en train de s'exécuter). Le **PID** est le numéro de badge de ce cuisinier : on peut lancer deux fois la même recette, on aura deux processus avec deux PID différents.

Observer les processus d'un conteneur

Lançons un conteneur nginx, puis vérifions qu'il fonctionne :

```
docker run -d --name linux-lab nginx
docker ps
```

Observer les processus à l'intérieur du conteneur :

```
docker top linux-lab
```

Docker stocke les métadonnées du conteneur dans son répertoire de données, généralement `/var/lib/docker/containers/<CONTAINER_ID>/` ; on y trouve notamment le fichier `config.v2.json`. Pour récupérer le **PID réel du conteneur sur l'hôte** :

```
docker inspect --format '{{.State.Pid}}' linux-lab

PID=$(docker inspect --format '{{.State.Pid}}' linux-lab)
ps -fp "$PID"
```

Un seul processus, deux numéros

Le processus principal du conteneur existe **sur l'hôte** avec un PID ordinaire (visible avec `ps`). C'est la preuve qu'un conteneur n'est qu'un processus Linux. La commande `docker top` montre d'ailleurs ces PID côté hôte, alors que le processus se croira **PID 1** à l'intérieur du conteneur (voir PID namespace, section 8.1).

À retenir

- ✓ Processus = programme en cours d'exécution ; PID = son identifiant.
- ✓ Un conteneur est visible sur l'hôte comme un **processus normal** (`docker inspect` donne son PID réel).

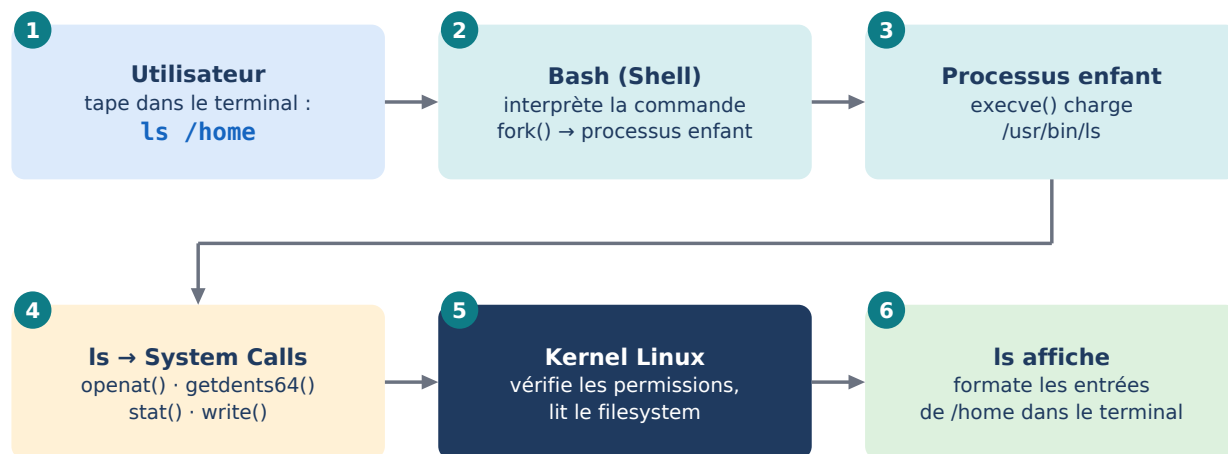
2. Le Kernel Linux

Le **kernel**, appelé **noyau** en français, est le cœur du système d'exploitation. C'est un programme logiciel de très bas niveau qui assure l'intermédiaire entre les programmes et les ressources de la machine.

En clair — Kernel et System Call

Imaginez un **hôtel**. Les clients (programmes) n'ont pas le droit d'entrer dans les cuisines, la salle des machines ou les chambres des autres. Ils passent par la **réception** (le **kernel**) et remplissent une **demande** (un **System Call**) : « ouvrir ce fichier », « envoyer ces données ». La réception vérifie que c'est autorisé, exécute, puis rend le résultat.

Que se passe-t-il quand on tape `ls /home` ?



Le kernel renvoie le résultat à `ls` : `ls` n'accède jamais directement au disque.

Figure 2 — De la commande `ls /home` jusqu'au kernel et retour à l'écran.

- L'utilisateur saisit la commande `ls /home` dans le terminal.
- **Bash** (le Shell) interprète la commande et doit lancer le programme `ls`. Il crée généralement un processus enfant avec `fork()`.
- Le processus enfant utilise `execve()` pour charger et exécuter `/usr/bin/ls`.
- **ls** ne communique pas directement avec le disque : il utilise des **System Calls** comme `openat()`, `getdents64()`, `stat()/statx()` et `write()` pour demander des services au kernel.
- Le **kernel** reçoit ces appels, vérifie notamment les **permissions** et utilise les mécanismes du filesystem pour accéder aux informations.
- Le kernel récupère les entrées de `/home` et retourne les informations au programme.
- Enfin, `ls` formate ces informations et les affiche dans le terminal.

À retenir

- ✓ Le **Shell** lance le programme, le programme utilise des **System Calls**, et le **Kernel** réalise les opérations sur les ressources du système.

Quelques System Calls importants

System Call	Rôle simplifié
<code>open()</code> / <code>openat()</code>	Ouvrir un fichier
<code>read()</code>	Lire des données
<code>write()</code>	Écrire des données
<code>close()</code>	Fermer un fichier
<code>fork()</code> , <code>execve()</code>	Créer un nouveau processus
<code>wait4()</code>	Attendre un processus

System Call	Rôle simplifié
<code>mount()</code>	Monter un filesystem
<code>unshare()</code>	Séparer un processus de certains namespaces
<code>chroot()</code>	Changer la racine apparente d'un processus
<code>pivot_root()</code>	Changer la racine du filesystem d'un environnement

Ces appels système sont particulièrement importants pour comprendre la création et l'isolation des conteneurs : `unshare()`, `mount()`, `chroot()` et `pivot_root()` sont exactement les outils qu'un runtime utilise pour fabriquer un conteneur.

En clair — `fork()` + `execve()`

`fork()` = faire une photocopie du processus en cours (le parent reste, un enfant apparaît). **`execve()`** = effacer le contenu de cette photocopie et le remplacer par un autre programme. Ensemble, ils permettent à bash de « lancer ls » : copie de bash, puis transformation de la copie en ls.

3. Filesystem

Un **filesystem** (système de fichiers) est le mécanisme permettant d'organiser et de gérer les fichiers et répertoires sur un système de stockage. Exemples : **ext4**, **XFS**, **Btrfs**. Linux présente les ressources sous forme d'une **arborescence** dont la racine est `/` :

```
/
├── bin/
├── etc/
├── home/
├── proc/
├── sys/
├── tmp/
├── usr/
└── var/
```

Observer la racine du conteneur :

```
docker exec linux-lab ls /
docker exec linux-lab ls /etc
docker exec linux-lab ls /usr
docker exec linux-lab ls /var
```

Le processus du conteneur voit une arborescence de fichiers qui constitue **son** environnement filesystem. Pour inspecter :

```
df -h
```

On voit un montage créé et utilisé automatiquement par Docker : c'est le filesystem du conteneur.

À retenir

- ✓ Le filesystem organise fichiers et répertoires en arborescence, à partir de la racine `/`.
- ✓ Le conteneur voit **sa propre** arborescence, différente de celle de l'hôte.

4. Root Filesystem : rootfs

Le **root filesystem**, souvent appelé **rootfs** dans le contexte des conteneurs, représente le système de fichiers visible comme racine `/` par un processus. Un rootfs minimal peut contenir par exemple :

```
rootfs/  
├─ bin/  
├─ etc/  
├─ lib/  
├─ proc/  
├─ tmp/  
├─ usr/  
└─ var/
```

Un conteneur n'a pas nécessairement besoin d'un système complet comme une machine virtuelle. Il peut utiliser un filesystem minimal contenant uniquement les fichiers nécessaires à son application.

En clair — rootfs

Le rootfs est un **appartement meublé** prêt à l'emploi : tout ce dont l'application a besoin est dedans (programmes, bibliothèques, fichiers de config). Le conteneur emménage dans cet appartement et ne voit rien en dehors. Attention : cet appartement ne contient **pas** de « fondations » (pas de kernel), car il utilise celles de l'immeuble (l'hôte).

À retenir

- ✓ rootfs = le système de fichiers vu comme / par le processus du conteneur.
- ✓ Il peut être très minimal : seulement ce que l'application exige.

5. Mount

Linux permet de **monter** un filesystem dans l'arborescence. Exemple :

```
mount /dev/sdb1 /mnt
```

Le contenu du filesystem devient alors accessible à partir de `/mnt`. On peut ainsi faire apparaître le même contenu à un autre endroit de l'arborescence. Les **mount namespaces** permettent ensuite à différents processus d'avoir des **vues différentes** de l'arborescence des montages.

Observer les montages faits automatiquement par le conteneur

```
df -h
```

Bind mount : monter un répertoire de l'hôte dans un conteneur

Les runtimes de conteneurs, comme Docker, permettent de monter des systèmes de fichiers ou des répertoires provenant de l'extérieur du système de fichiers du conteneur, notamment depuis le système hôte.

1) Créer un répertoire et un fichier sur l'hôte :

```
mkdir -p /tmp/container-data  
echo "hello from host" > /tmp/container-data/test.txt
```

2) Créer un conteneur avec un bind mount :

```
docker run -d --name linux-lab-mount \  
  --mount type=bind,src=/tmp/container-data,dst=/data \  
  nginx  
  
# Syntaxe équivalente, plus courte :  
docker run -d --name linux-lab-mount3 -v /tmp/container-data:/data:Z nginx
```

3) Observer le fichier depuis le conteneur, puis inspecter :

```
docker exec linux-lab-mount cat /data/test.txt
docker inspect --format '{{json .Mounts}}' linux-lab-mount
docker exec linux-lab-mount findmnt
```

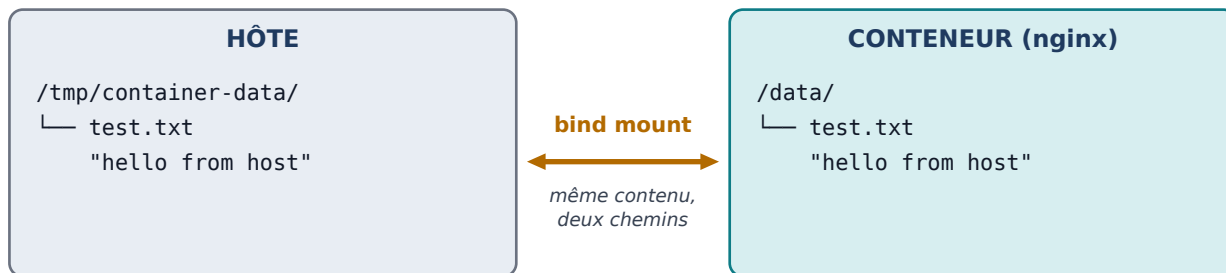


Figure 3 — Un bind mount : le même contenu est visible à deux endroits différents.

Le même contenu présent sur l'hôte devient accessible dans le conteneur à travers `/data`.

À quoi sert « :Z » ?

Le suffixe `:Z` de la syntaxe `-v` sert aux systèmes qui utilisent **SELinux** (Fedora, RHEL...) : il demande de réétiqueter le répertoire pour que le conteneur ait le droit de l'utiliser. Sur un système sans SELinux, il est sans effet.

À retenir

- ✓ Monter = rendre un filesystem accessible à un endroit de l'arborescence.
- ✓ Un **bind mount** expose un dossier de l'hôte dans le conteneur.
- ✓ Le **mount namespace** donne à chaque conteneur sa propre vue des montages.

6. chroot

`chroot` signifie *change root*. Il permet de modifier la **racine apparente** (`/`) d'un processus. Par exemple :

```
chroot /my-rootfs
```

Après cette opération, le processus considère `/my-rootfs` comme sa nouvelle racine `/`.

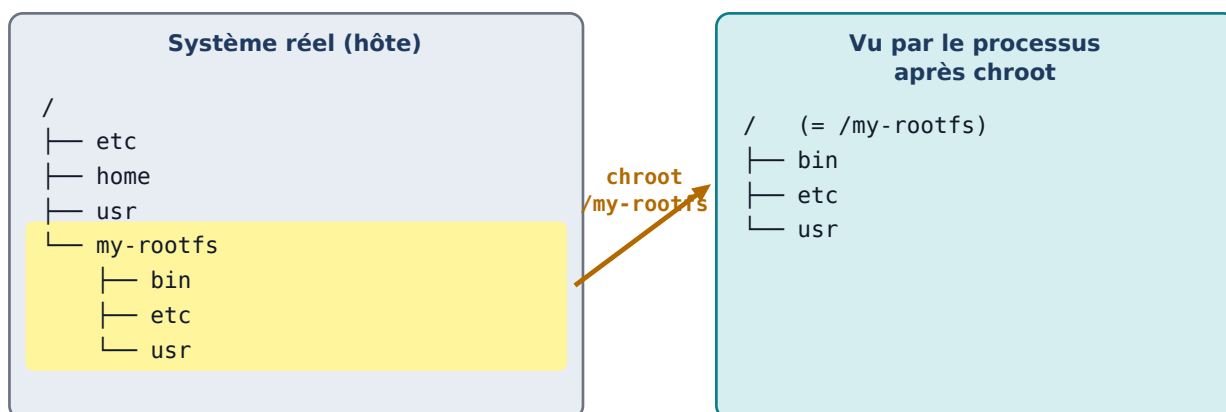


Figure 4 — Avec `chroot`, le processus voit `/my-rootfs` comme s'il s'agissait de la racine.

En clair — `chroot`

C'est comme **changer le panneau « Entrée »** d'un bâtiment : le processus croit que la pièce `/my-rootfs` est tout le bâtiment, mais le vrai bâtiment est toujours là, avec le même réseau, les mêmes processus et sans aucune limite de ressources.

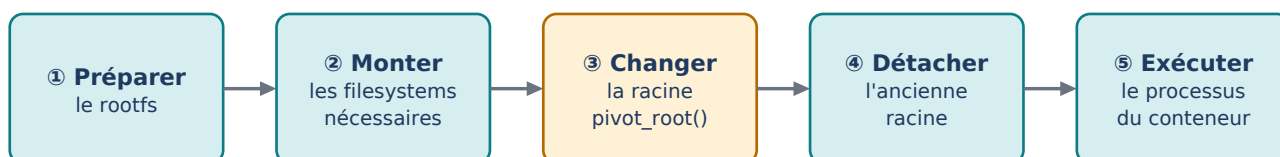
Attention

`chroot` ne suffit **pas** à créer un conteneur complet : il ne change que la racine apparente. Le processus partage toujours les mêmes processus visibles, le même réseau, le même hostname et les mêmes utilisateurs que l'hôte.

7. pivot_root

`pivot_root()` permet de changer la racine du filesystem d'un environnement et de **déplacer l'ancienne racine** vers un autre emplacement. Il est particulièrement pertinent lorsqu'on construit un environnement de filesystem isolé.

Dans une construction manuelle de conteneur, on rencontre une séquence conceptuelle comme :



Contrairement à `chroot`, l'ancienne racine est déplacée puis détachée : le processus ne peut plus y revenir.

Figure 5 — Séquence conceptuelle de construction d'un filesystem isolé.

`pivot_root` est donc plus adapté à une véritable construction d'environnement isolé que le simple changement de racine fourni par `chroot`.

	chroot	pivot_root
Principe	Change la racine apparente	Change la racine et déplace l'ancienne
Ancienne racine	Reste accessible sur le système	Peut être détachée (démontée)
Usage typique	Simple, pratique pour tester	Construction d'un vrai environnement isolé

8. Les Namespaces Linux

Les **Linux Namespaces** permettent d'**isoler la vue** qu'un processus possède de certaines ressources du système. Une façon simple de retenir leur rôle :

Namespace = « Qu'est-ce que le processus peut voir ? »

En clair — Namespace

Un namespace, ce sont des **lunettes spéciales** (ou une pièce sans fenêtre sur le reste de la maison). Les ressources existent toujours sur la machine, mais le processus ne voit que celles « à sa portée ». Il y a une paire de lunettes par type de ressource : une pour les processus, une pour le réseau, etc.

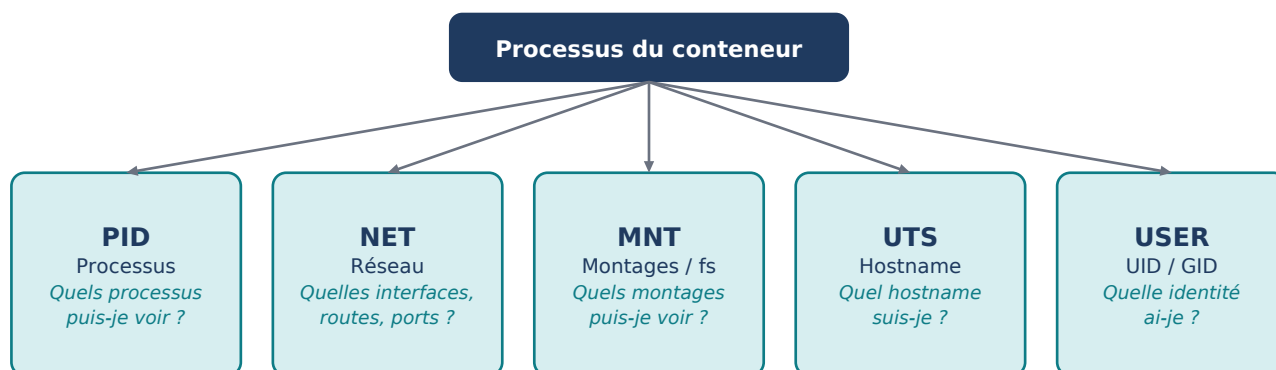


Figure 6 — Un conteneur combine plusieurs namespaces ; chacun répond à une question différente.

8.1. PID Namespace

Le **PID namespace** isole la vue des processus. Un processus dans un PID namespace peut avoir une vue différente des processus présents sur l'hôte. Le processus principal du conteneur peut donc apparaître comme **PID 1** à l'intérieur du conteneur.

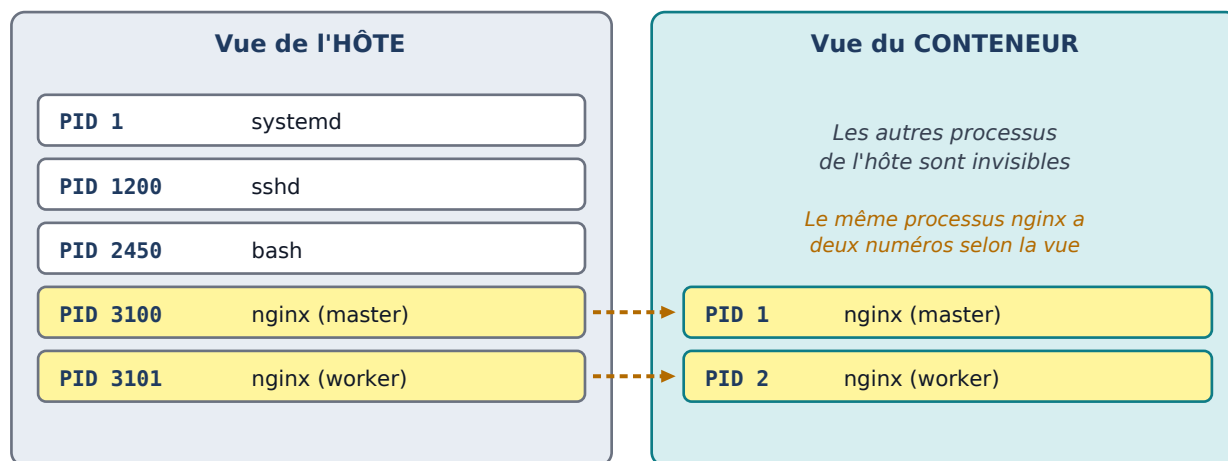


Figure 7 — Le même processus nginx : PID 3100 sur l'hôte, PID 1 dans le conteneur.

8.2. Network Namespace

Le **Network namespace** permet d'isoler l'environnement réseau. Il peut isoler notamment : **interfaces réseau**, **adresses IP**, **routes**, **tables de routage** et **ports**. Les mécanismes comme **veth**, les **bridges** et le **NAT** permettent ensuite de connecter ces namespaces au réseau extérieur.

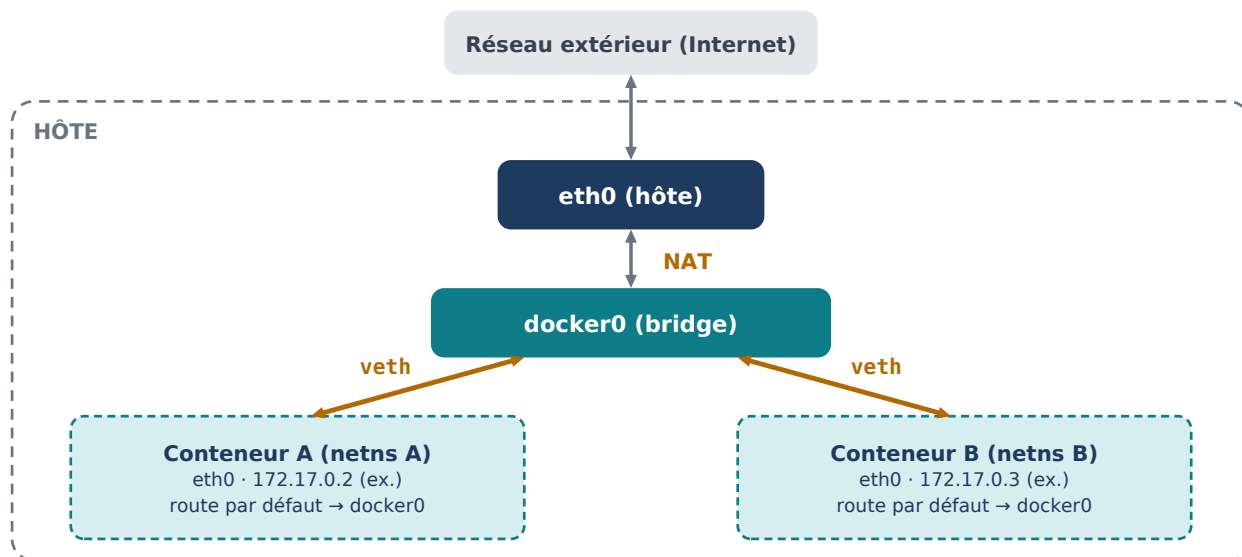


Figure 8 — Chaque conteneur a sa pile réseau ; veth + bridge + NAT les relient au monde extérieur.

En clair — veth, bridge, NAT

veth = un câble réseau virtuel à deux extrémités (une dans le conteneur, une sur l'hôte). **Bridge** (ex. `docker0`) = un switch virtuel qui relie plusieurs câbles. **NAT** = le standardiste qui traduit les adresses privées du conteneur en l'adresse de l'hôte pour sortir sur Internet.

Observer les interfaces réseau du conteneur :

```
docker run -d --name mon_reseau nicolaka/netshoot sleep infinity
docker exec -it mon_reseau ip a

# Observer les routes
docker exec mon_reseau ip route
```

Inspection avec Docker, puis affichage direct de l'IP :

```
docker inspect --format '{{json .NetworkSettings}}' mon_reseau

docker inspect --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' mon_reseau
```

Inspection du namespace :

```
PID=$(docker inspect --format '{{.State.Pid}}' mon_reseau)
ls -l /proc/"$PID"/ns/net
```

Le conteneur possède sa propre vue des interfaces réseau, des adresses IP et des routes.

Astuce de lecture

Le lien affiché par `ls -l /proc/$PID/ns/net` ressemble à `net:[4026532xxx]`. Ce nombre identifie le namespace : deux processus qui affichent le **même** nombre partagent le même namespace réseau. Comparez avec celui d'un processus de l'hôte : ils seront différents.

8.3. Mount Namespace

Le **Mount namespace** permet à un processus d'avoir sa propre vue des montages. C'est un mécanisme fondamental pour construire le **filesystem isolé** d'un conteneur.

Host	Container
<pre> / ├─ etc ├─ home ├─ var └─ data </pre>	<pre> / ├─ etc ├─ app ├─ proc └─ tmp </pre>

8.4. UTS Namespace

Le **UTS namespace** permet notamment d'isoler le **hostname** (nom de la machine). Le système hôte peut avoir `server01` alors qu'un conteneur a `web-container`.

Observer le hostname dans le conteneur, comparer avec l'hôte, puis inspecter :

```

docker exec linux-lab hostname
hostname
docker inspect --format '{{.Config.Hostname}}' linux-lab

```

Généralement, le hostname par défaut du conteneur est son **ID** (ses premiers caractères). Le conteneur peut donc avoir un hostname différent de celui de l'hôte grâce au UTS namespace.

8.5. User Namespace

Le **User namespace** permet d'isoler les identifiants utilisateurs et groupes (**UID/GID**). C'est particulièrement important pour le **rootless container**. Un processus peut par exemple être **UID 0** à l'intérieur d'un namespace tout en correspondant à un utilisateur **non-root** sur l'hôte. Cela permet de **réduire les privilèges** nécessaires à l'exécution de certains conteneurs.

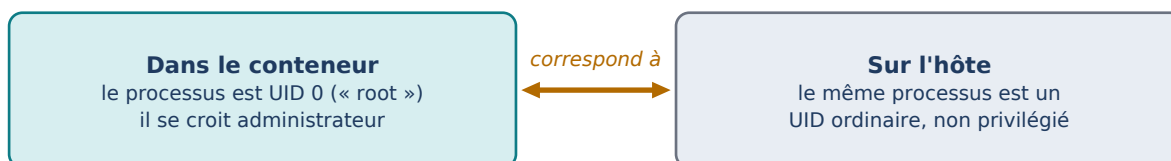


Figure 9 — User namespace : « root » dans le conteneur, simple utilisateur sur l'hôte.

En clair — Rootless

Un conteneur **rootless** est lancé par un utilisateur ordinaire, sans droits d'administrateur sur l'hôte. Même si une personne malveillante devient « root » dans le conteneur, elle n'a que les droits d'un utilisateur ordinaire sur la machine : c'est comme avoir un badge « directeur » valable seulement dans une salle, et un simple badge visiteur dans le reste du bâtiment.

8.6. Résumé des Namespaces

Namespace	Ce qu'il isole	Question à retenir
PID	Processus	Quels processus puis-je voir ?
NET	Réseau	Quelles interfaces/routes puis-je voir ?
MNT	Montages / filesystem	Quels montages puis-je voir ?
UTS	Hostname	Quel hostname suis-je ?
USER	UID/GID	Quelle identité utilisateur ai-je ?

Pour aller plus loin : Linux possède aussi des namespaces IPC (communication entre processus), Cgroup et Time, non détaillés dans ce chapitre. Un processus rejoint ou crée des namespaces avec les appels système `unshare()` et `clone()`.

À retenir

- ✓ Namespace = isolation de la **visibilité** (ce que le processus voit).
- ✓ Un conteneur combine plusieurs namespaces : PID, NET, MNT, UTS, USER.
- ✓ Le processus principal du conteneur est **PID 1** dans son PID namespace, mais a un autre PID sur l'hôte.

9. Cgroups

Les **Control Groups**, ou **cgroups**, permettent de **contrôler et de limiter** les ressources utilisées par des groupes de processus. Une façon simple de retenir leur rôle :

Cgroup = « Combien de ressources le processus peut-il utiliser ? »

Mécanisme	Rôle
Namespaces	Isolation / visibilité
Cgroups	Contrôle / limites de ressources

En clair — Cgroup

Un cgroup, c'est un **forfait téléphonique** : le processus peut utiliser des données, des minutes et des SMS, mais seulement jusqu'au plafond du forfait. Les namespaces décident ce que le processus *voit* ; le cgroup décide ce qu'il peut *consommer*.

9.1. Limiter le CPU

Le **CPU** exécute les instructions des programmes. Plus un programme dispose de temps CPU, plus il peut effectuer de travail rapidement, toutes choses égales par ailleurs. Dans un environnement conteneurisé, on peut limiter ou contrôler la quantité de CPU utilisable par un groupe de processus : Container A, B et C ont chacun leur limite CPU.

9.2. Mémoire

La **mémoire RAM** conserve les données et programmes nécessaires à leur exécution. Un conteneur peut être soumis à une limite mémoire (par exemple `Memory limit = 512 MB`). Cela empêche un processus ou un groupe de processus de consommer une quantité illimitée de mémoire.

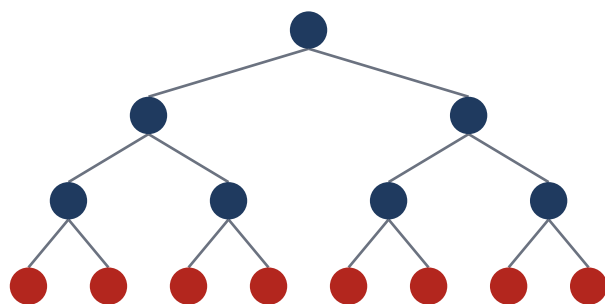
Que se passe-t-il en cas de dépassement ?

Si un conteneur dépasse sa limite mémoire, le kernel peut tuer un de ses processus (**OOM kill**, *Out Of Memory*). Dépassez une limite CPU, en revanche, ne tue rien : le processus est simplement **ralenti**.

9.3. Limiter les PIDs

Les cgroups peuvent également limiter le **nombre de processus** qu'un groupe peut créer (par exemple `PID limit = 100`). Cela contribue à limiter certains comportements excessifs ou certaines attaques de type **fork bomb**.

Une **fork bomb** (bombe fork) est un type d'attaque par **déni de service (DoS)** qui consiste à forcer un système à dupliquer un processus de manière récursive et infinie pour saturer ses ressources.



Chaque processus se duplique : 1 → 2 → 4 → 8 → ...

Figure 10 — Une fork bomb et l'effet d'une limite de PIDs.

Sans limite

duplication infinie → saturation de l'hôte (déni de service)

Avec --pids-limit=50

fork() refusé au-delà de la limite → l'hôte reste stable

Exemple : comment Docker utilise les cgroups

Les runtimes comme Docker s'appuient sur les cgroups pour appliquer ces limitations. Docker fournit des options pour définir ces limites à la création du conteneur, sans manipuler directement les cgroups. Créons un conteneur avec plusieurs limites :

```
docker run -d --name linux-lab-limited \
  --memory=128m --cpus=0.5 --pids-limit=50 nginx
```

Inspection des limites, puis observation en fonctionnement :

```
docker inspect linux-lab-limited \
  --format 'Mem={{.HostConfig.Memory}} CPU={{.HostConfig.NanoCpus}} PIDs={{.HostConfig.PidsLimit}}'
docker stats linux-lab-limited
```

On peut notamment observer : CPU %, MEM USAGE / LIMIT, MEM % et PIDS.

```
docker run --memory=128m --cpus=0.5 --pids-limit=50 nginx
```

CPU

temps processeur

≈ 0,35 cœur utilisé

limite : 0,5 cœur

(= 50 % d'un cœur)

Mémoire

RAM

≈ 80 Mo utilisés

limite : 128 Mo

au-delà → OOM kill

PIDs

nb de processus

15 / 50

limite : 50 processus

au-delà → fork() échoue

Cgroup = « combien de ressources le processus peut-il utiliser ? »

Figure 11 — Les trois limites de la commande docker run et ce qui se passe au plafond (valeurs d'usage illustratives).

9.4. Résumé des Cgroups

Ressource	Exemple de contrôle
CPU	Limiter ou pondérer l'utilisation CPU
Mémoire	Définir une limite mémoire
PIDs	Limiter le nombre de processus

À retenir

- ✓ Cgroup = contrôle des **quantités** de ressources (CPU, mémoire, PIDs).
- ✓ Namespaces = **isolation** ; Cgroups = **limites**. Les deux sont complémentaires.
- ✓ Avec Docker : `--memory`, `--cpus`, `--pids-limit` configurent les cgroups à votre place.

10. Exemple complet : lancement d'un conteneur

Lorsque l'utilisateur exécute par exemple `docker run nginx`, la logique générale est beaucoup plus complexe, mais on peut la simplifier ainsi :

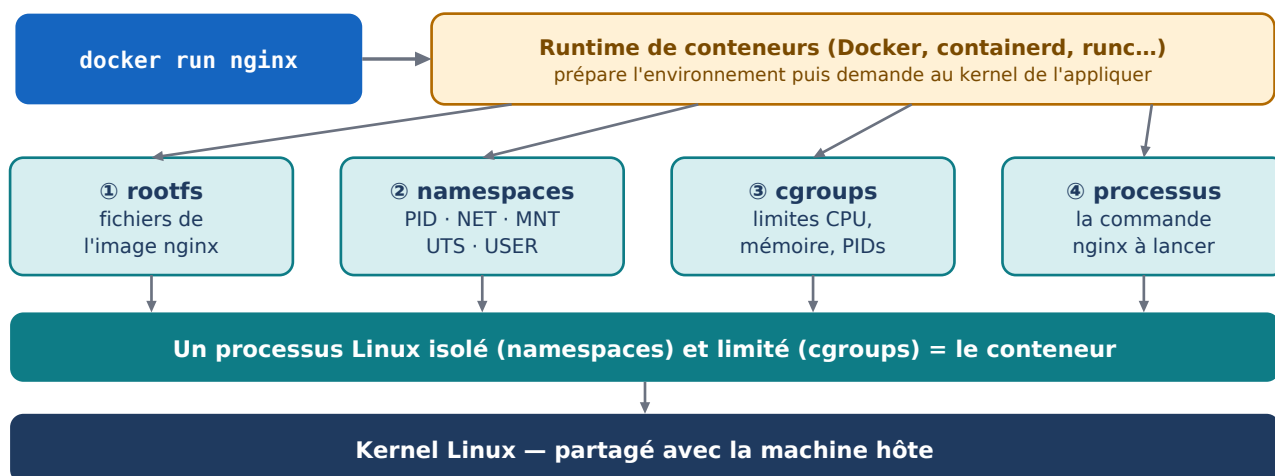


Figure 12 — Simplification de `docker run` : `rootfs` + `namespaces` + `cgroups` + `processus`, au-dessus du kernel.

Le résultat est **un processus Linux qui s'exécute dans un environnement isolé**. Maintenant on comprend réellement la différence entre **conteneurisation** et **virtualisation** : un conteneur partage le même OS principal (le même kernel).

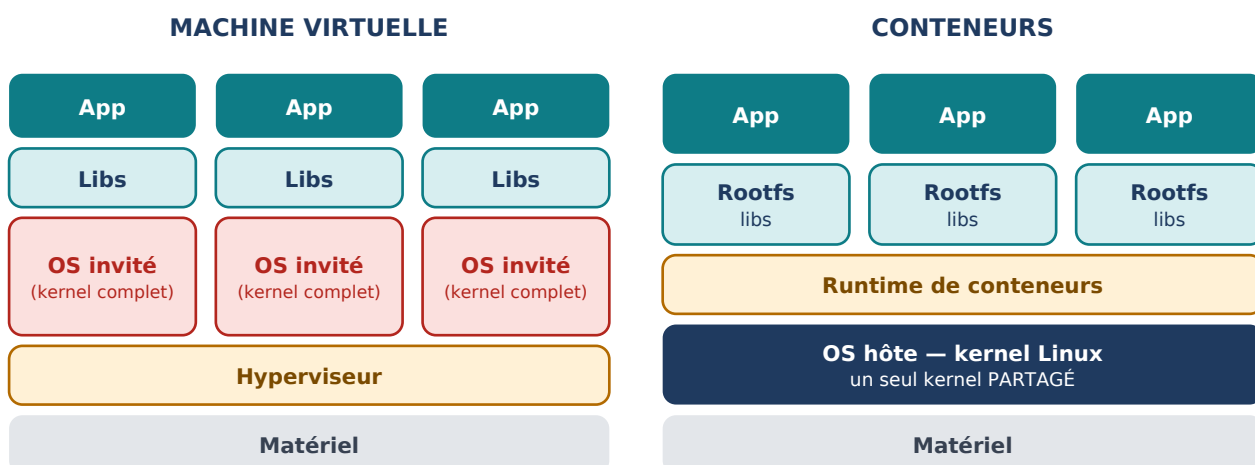


Figure 13 — Machine virtuelle (un OS complet par VM) contre conteneurs (un seul kernel partagé).

Critère	Machine virtuelle	Conteneur
Kernel	Un kernel complet par VM	Kernel de l'hôte partagé
Démarrage	Plus lent (boot d'un OS)	Très rapide (lancement d'un processus)
Poids	Lourd (Go)	Léger (rootfs minimal)
Isolation	Forte (matériel virtualisé)	Logique (namespaces + cgroups)

Conséquence du kernel partagé

Comme les conteneurs partagent le kernel de l'hôte, un conteneur Linux a besoin d'un kernel Linux. Sous Windows ou macOS, Docker lance donc une petite machine virtuelle Linux en arrière-plan. Ce partage du kernel explique aussi pourquoi l'isolation est moins étanche que celle d'une VM.

Ce qu'il faut absolument retenir

Kernel

Le cœur du système d'exploitation. Il gère les ressources et fournit des services aux programmes.

Shell

Un programme qui interprète les commandes de l'utilisateur et lance d'autres programmes.

System Call

Une interface permettant à un programme de demander un service au kernel.

Processus

Un programme en cours d'exécution.

fork()

Permet de créer un nouveau processus. Il ne constitue pas à lui seul un mécanisme d'isolation de conteneur.

Filesystem

Organise les fichiers et répertoires du système.

Rootfs

Le système de fichiers visible comme / par un environnement.

chroot

Change la racine apparente d'un processus, mais ne suffit pas à créer un conteneur complet.

pivot_root

Permet de changer la racine d'un environnement de filesystem.

Namespace

Isole la vue d'un processus sur certaines ressources du système.

Cgroup

Permet de contrôler les ressources utilisées par un groupe de processus.

La synthèse en 4 phrases

1. Un conteneur est un **processus Linux**, pas une VM.
2. Les **namespaces** décident ce qu'il **voit**.
3. Les **cgroups** décident ce qu'il peut **consommer**.
4. Le **rootfs** lui fournit ses fichiers, et le **kernel** (partagé avec l'hôte) fait tout fonctionner.

Glossaire des termes importants

Terme	Définition simple
Hôte (host)	La machine physique ou virtuelle sur laquelle tourne Docker et son kernel.
PID	Numéro unique d'un processus. Un même processus peut avoir un PID différent sur l'hôte et dans son PID namespace.
execve()	Appel système qui remplace le programme d'un processus par un autre programme.
System Call	Demande formelle d'un programme au kernel (ouvrir un fichier, créer un processus...).
Espace utilisateur / noyau	Les programmes tournent avec des droits limités (espace utilisateur) ; seul le kernel a un accès complet à la machine.
Mount / montage	Action de rendre un filesystem accessible à un endroit de l'arborescence.
Bind mount	Montage qui expose un répertoire existant (par ex. de l'hôte) à un autre emplacement, par ex. dans un conteneur.
overlay / union fs	Superposition de couches de fichiers : technique habituellement utilisée par Docker pour assembler le filesystem d'un conteneur.
UTS	Namespace du hostname (nom historique : <i>UNIX Time-sharing System</i>).
UID / GID	Identifiants numériques d'un utilisateur / d'un groupe. UID 0 = root.
Rootless	Conteneur exécuté sans privilèges root sur l'hôte, grâce notamment au user namespace.
veth	Paire d'interfaces réseau virtuelles reliées comme les deux bouts d'un câble.
Bridge	Switch virtuel qui relie plusieurs interfaces (ex. <code>docker0</code>).
NAT	Traduction d'adresses : permet aux conteneurs d'accéder à l'extérieur avec l'adresse de l'hôte.
OOM kill	Arrêt forcé d'un processus par le kernel lorsqu'un groupe dépasse sa limite de mémoire.
Fork bomb	Attaque par déni de service : un processus se duplique à l'infini pour saturer la machine.
DoS	Déni de service : saturer une machine pour la rendre inutilisable.
Runtime de conteneurs	Logiciel (Docker, containerd, runc...) qui prépare rootfs, namespaces et cgroups puis lance le processus du conteneur.
Virtualisation	Émulation de machines complètes (chacune avec son propre OS) au-dessus d'un hyperviseur.
Conteneurisation	Isolation de processus qui partagent le kernel de l'hôte.

Questions de compréhension — avec corrigés

Essayez de répondre de vous-même avant de lire la correction.

Question 1 Quelle est la différence entre le Shell et le Kernel ?

Correction

Le **Shell** (ex. `bash`) est un programme en espace utilisateur : il lit les commandes tapées, puis lance les programmes correspondants (via `fork()` et `execve()`). C'est une **interface** entre l'utilisateur et le système.

Le **Kernel** est le cœur privilégié du système d'exploitation : il gère réellement les ressources (processus, mémoire, fichiers, réseau) et vérifie les permissions.

Exemple : avec `ls /home`, le Shell lance `ls`, mais c'est le kernel qui lit le répertoire.

Question 2 Pourquoi une application utilise-t-elle des System Calls ?

Correction

Une application n'a pas le droit d'accéder directement au matériel ni aux ressources des autres programmes (sécurité et stabilité). Les **System Calls** sont l'unique porte d'entrée pour **demandeur un service au kernel** : ouvrir un fichier, créer un processus, envoyer des données sur le réseau.

Le kernel vérifie alors les permissions et exécute l'opération à sa place.

Question 3 Que fait `fork()` ?

Correction

Il **crée un nouveau processus** (l'enfant), copie du processus appelant, avec un nouveau PID. Il est généralement suivi de `execve()` pour charger un autre programme dans l'enfant.

Question 4 Pourquoi `fork()` seul ne suffit-il pas à créer un conteneur ?

Correction

Car `fork()` ne fait que **dupliquer** un processus. L'enfant a les mêmes vues que l'hôte (mêmes processus, même réseau, même racine, mêmes utilisateurs) et aucune limite de ressources.

Pour en faire un conteneur, il manque l'**isolation** (namespaces), la **limitation** (cgroups) et un **rootfs** dédié.

Question 5 À quoi sert un PID namespace ?

Correction

Il **isole la vue des processus** : un processus ne voit que ceux de son namespace, et le premier y reçoit le PID 1. Les processus de l'hôte lui sont invisibles, alors que l'hôte voit le processus du conteneur avec un autre PID.

Question 6 À quoi sert un Network namespace ?

Correction

Il donne au conteneur sa **propre pile réseau** : interfaces, adresses IP, routes et ports. Deux conteneurs peuvent par exemple écouter chacun sur le port 80 sans conflit. Des veth, un bridge et du NAT le relie ensuite au réseau extérieur.

Question 7 À quoi sert un Mount namespace ?

Correction

Il donne au processus sa **propre vue des points de montage**. C'est la base du filesystem isolé d'un conteneur : on peut y monter rootfs, `/proc` ou des volumes sans modifier les montages de l'hôte.

Question 8 Quelle est la différence entre un Namespace et un Cgroup ?**Correction**

Namespace = **visibilité** (« qu'est-ce que je peux voir ? »). **Cgroup** = **quantité de ressources** (« combien puis-je consommer ? »).

Ils sont complémentaires : avec les seuls namespaces, un conteneur est isolé mais peut saturer la RAM ; avec les seuls cgroups, il est limité mais voit tout le système.

Question 9 Pourquoi chroot seul n'est-il pas équivalent à un conteneur ?**Correction**

chroot ne change que la **racine apparente** du filesystem. Il n'isole ni les processus, ni le réseau, ni le hostname, ni les utilisateurs, et ne limite aucune ressource. En outre, l'ancienne racine n'est pas détachée, ce qui rend l'évasion possible dans certains cas : **pivot_root** avec un mount namespace est plus adapté.

Question 10 Quel rôle joue le rootfs ?**Correction**

Il fournit à l'application **son environnement de fichiers** : programmes, bibliothèques et configuration, vus comme **/**. Il peut être minimal (juste ce que l'application exige) et ne contient pas de kernel.

Question 11 Quel est le rôle du kernel dans l'architecture d'un conteneur ?**Correction**

Le kernel **fournit tous les mécanismes** : namespaces, cgroups, montages, réseau, permissions. Le runtime (Docker...) ne fait que les demander. Comme le kernel est **partagé** avec l'hôte, les conteneurs sont légers et rapides, mais ils dépendent tous du même kernel.

Préparation pour la suite

Après avoir compris ces primitives Linux, nous pouvons passer à la **construction manuelle d'un conteneur**. La prochaine étape combine concrètement :



Rootfs + Namespaces + Cgroups + Process = Mini-conteneur

L'objectif du lab suivant est de construire progressivement cet environnement **sans commencer directement par Docker**, afin de comprendre ce que les outils de conteneurisation automatisent réellement. Le lab est disponible dans le dépôt du guide :

github.com/halekammoun/Guide-complet-sur-la-conteneurisation (fichier TPs/TP-01.md).

Aide-mémoire des commandes

Commande	À quoi elle sert
<code>ps aux / top</code>	Lister / surveiller les processus
<code>docker top <nom></code>	Processus d'un conteneur (PID vus côté hôte)
<code>docker inspect --format '{{.State.Pid}}' <nom></code>	PID réel du conteneur sur l'hôte

Commande	À quoi elle sert
<code>df -h</code>	Voir les montages et l'espace disque
<code>findmnt</code>	Afficher l'arbre des montages
<code>docker run -v hôte:cible</code>	Bind mount d'un dossier de l'hôte
<code>ls -l /proc/\$PID/ns/</code>	Identifier les namespaces d'un processus
<code>ip a / ip route</code>	Interfaces et routes réseau (dans le conteneur)
<code>hostname</code>	Nom de la machine (UTS namespace)
<code>docker run --memory --cpus --pids-limit</code>	Définir des limites cgroups
<code>docker stats <nom></code>	Consommation et limites en temps réel